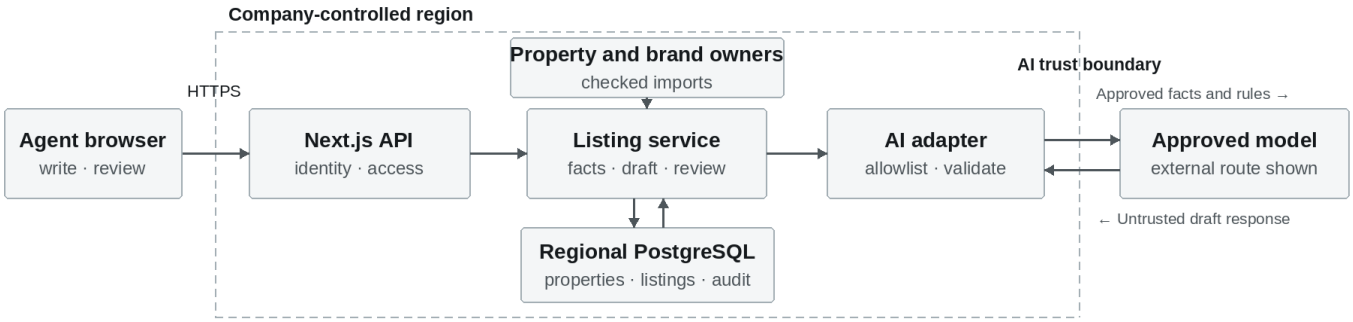
Part B Architecture and design

Use a Next.js modular monolith for the six-week pilot in one country and language. Agents write listings manually, with optional AI description suggestions help, then review the wording. Source access, staffing, country rules, processing approval, and traffic targets still need confirmation.

System and trust boundaries:



- A Property holds physical facts.
- A Listing belongs to a property and owns its marketing title, description, rent or sale purpose, price, currency, and status. One property may have several listings.
- Agents write manually or request an AI proposal, which they must explicitly apply. The server authorizes access and selects processing policy from the property country.

Only approved facts and brand rules cross the model boundary. The adapter validates the response, and the UI shows it as plain text beside the facts. An approved internal model can replace external processing.

ADR 1 Modular monolith:

Context and options. Six weeks, one workflow, and an assumed team of two developers. Consider an undivided backend, a modular monolith, or separate services.

Decision. Deploy the Next.js UI and API together, backed by PostgreSQL. Separate property facts, listing and review rules, HTTP handling, and model adapters. Use server-side search, typed filters, sorting, and pagination. Keep domain rules independent of HTTP, SQL, and vendor SDKs.

Trade-off. One deployment simplifies releases and transactions, but modules share scaling and release schedules. Forty thousand records alone do not justify services. Load-test before adding replicas. Extract a service when isolation or independent scaling is demonstrably needed.

ADR 2 Application owned AI contract:

Context and options. Providers and deployment models may change, and model behavior differs. Consider direct SDK calls, a generic AI gateway, or a narrow listing-generation contract.

Decision. Inject an adapter that accepts approved facts, brand rules, and locale, then returns a validated draft and normalized usage or errors. Keep credentials, vendor parsing, and transport inside adapters. Server policy selects the approved route.

Trade-off. Business rules remain vendor-independent. Each adapter still needs processing approval and quality regression tests.

ADR 3 Checked facts and processing policy:

Context and options. Source APIs and processing permissions are unresolved. Consider full live records, checked structured snapshots, or a broader synchronized property platform.

Decision. Start with an approved import. Agree a source owner, refresh cadence, and freshness limit. Allowlist beds, baths, size, location, and amenities. Exclude resident and prospect data, internal notes, and identifiers. Version facts, brand rules, prompts, and policy. Block generation when facts are missing or stale, or processing is unapproved.

Trade-off. Checked imports limit disclosure but need refresh ownership. Save fact and policy versions with reviewed listing revisions. Changed wording or property facts invalidate review on the server, in the same transaction as the update. Reject stale writes with HTTP 409. Changed brand rules or policy also require renewed review.

Patterns and deliberate omissions:

- Use ports and adapters for property access and AI calls, so either integration can change.
- Inject adapters to test domain rules through the same contract.
- Parse requests, imports, and model responses at their boundaries, then let internal rules use validated types.
- Omit CQRS, an event bus, vector search, and an agent framework. Selected-property drafting needs none of them.

Privacy and access:

- Compliance confirms applicable GDPR obligations and approves fields, provider retention and training terms, and residency before live use.
- Apply residency rules to the application, database, logs, backups, and model processing.
- Enforce organization, country, and property access on the server.
- Keep keys server-side, encrypt transport and storage, minimize retention, and support agreed deletion.

AI quality and human review:

- Treat property strings as untrusted data. Separate them from model instructions and give the model no tools. Validate output shape and length, render plain text, and check basic factual consistency. These checks cannot establish truth.
- Use approved, versioned brand examples.
- Require agent review and Marketing sampling.
- Evaluate factual errors, prohibited claims, brand acceptance, and Part A's time and draft-acceptance measures on representative listings.

Cost performance and reliability:

- Set user and organization quotas, a global concurrency limit, token caps, and request deadlines.
- Cache reference data by version. Initially let agents explicitly reuse saved drafts rather than cache model responses.
- Preserve inputs after a timeout or invalid output and offer manual drafting or an explicit retry.
- Avoid automatic retries when completion is uncertain.
- Agree latency and cost targets, then load-test expected pilot concurrency.

Observability:

- Record request and draft references, source versions, provider and model, latency, token use, estimated cost, and safe error codes.
- Audit access and review events under approved retention.
- Keep raw prompts, property payloads, and draft bodies out of routine logs.
- Use approved, access-controlled samples for quality review.
- Alert on failures, spending, and quality trends.

Country and language expansion:

- Configure languages, brand rules, and approved model routes per country. Derive country from authorized property data, not browser locale.
- Reuse the application in regional deployments with isolated stores and backups where required.
- Add source connectors gradually and index property access by organization, country, and property ID.
- Check each market's rules and language quality before enabling it.

Team and delivery

- With two developers, assign the UI and review flow, domain and AI work, and data, security, and delivery. A fourth can own deployment and evaluation.
- Agree contracts together. First deliver checked facts through a real model to a validated draft the agent can review.
- Week 1 resolves permissions, source access, and the baseline.
- Weeks 2 and 3 build the pilot.
- Week 4 covers quality, security, and load checks.
- Week 5 introduces the cohort.
- Week 6 measures results and decides expansion.